MODÈLE PHYSIQUE DE DONNÉES

KotBis / MoBis - Bus Maurice

Version 0.3 - Corrigé selon le cahier des charges
PostgreSQL 16 + PostGIS 3.4

Version	0.3 - Copie corrigée de la v0.2 pour conformité au CDC
Date	11/11/2026
Statut	Provisoire - En attente de validation
Référence	Cahier des charges Back-Office & Base de données MoBis v1.0 (29/10/2026), §6 et règles RG01-RG12
Format	Représentation MPD texte + CREATE TABLE

Règle appliquée : aucune table, colonne, contrainte ou règle n'a été ajoutée si elle n'est pas dans le CDC. Les éléments que le CDC n'a pas précisés sont signalés comme tels (voir section 8).

1. Introduction

Ce document présente le **Modèle Physique de Données (MPD)** de KotBis, corrigé pour que ses 18 tables correspondent à celles du CDC (§6.1). Pour chaque table : (1) représentation MPD en texte brut, (2) script SQL. Les colonnes géographiques utilisent PostGIS (geometry Point/Polygon, SRID 4326). Archivage via colonne **archive_le** (timestamptz NULL) sur les objets réseau. Concurrence optimiste via colonne **version**.

2. Liste des tables

#	Table	Domaine	Description
1	**roles**	Identité	Rôles applicatifs
2	**comptes**	Identité	Connexion unique (email + hash)
3	**personnel**	Identité	Profils agents back-office
4	**voyageurs**	Identité	Profils voyageurs
5	**sessions**	Identité	Sessions serveur (empreinte du jeton)
6	**lignes**	Réseau	Lignes de bus
7	**bus**	Réseau	Véhicules
8	**arrets**	Réseau	Arrêts géolocalisés (PostGIS)
9	**trajets**	Réseau	Trajets (sens)
10	**trajet_arrets**	Réseau	Étapes ordonnées
11	**zones**	Réseau	Zones polygones (PostGIS)
12	**affectations_bus**	Réseau	Historique bus / trajet
13	**signalements**	Contributions	Signalements voyageurs
14	**historique_signalements**	Contributions	Décisions signalements
15	**avis**	Contributions	Avis voyageurs
16	**historique_avis**	Contributions	Décisions modération
17	**mouvements_points**	Points/Audit	Journal points (append-only)
18	**journal_audit**	Points/Audit	Audit technique

3. Détail des tables - MPD + SQL

3.1. Table ROLES

Référentiel des rôles (voyageur, operateur, moderateur, administrateur).

1. Représentation MPD (texte brut)

```
+----------------------------------------+
|                 ROLES                  |
+----------------------------------------+
| PK id                   : SMALLINT     |
|    code                 : VARCHAR(30)  |
|    libelle              : VARCHAR(80)  |
+----------------------------------------+
```

2. Script SQL

```
CREATE TABLE roles (
    id                  SMALLINT PRIMARY KEY,
    code                VARCHAR(30) NOT NULL UNIQUE,
    libelle             VARCHAR(80) NOT NULL,
    CONSTRAINT chk_roles_code CHECK (code IN ('voyageur','operateur','moderateur','administrateur'))
);
```

Référentiel initial du CDC §6.6 ; code NN, UQ, liste de codes approuvée (CHECK).

3.2. Table COMPTES

Identité de connexion commune au personnel et aux voyageurs.

1. Représentation MPD (texte brut)

```
+-----------------------------------------------+
|                   COMPTES                     |
+-----------------------------------------------+
| PK id                   : BIGINT (identity)   |
|    email                : VARCHAR(254)        |
|    mot_de_passe_hash    : TEXT                |
| FK role_id              : SMALLINT            |
|    etat                 : VARCHAR(20)         |
|    identifiant_public   : VARCHAR(40)         |
|    invitation_hash      : TEXT                |
|    invitation_expire_le : TIMESTAMPTZ         |
|    version              : INTEGER             |
|    cree_le              : TIMESTAMPTZ         |
|    modifie_le           : TIMESTAMPTZ         |
+-----------------------------------------------+
```

2. Script SQL

```
CREATE TABLE comptes (
    id                  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email               VARCHAR(254) NOT NULL,
    mot_de_passe_hash   TEXT,
    role_id             SMALLINT NOT NULL REFERENCES roles(id),
    etat                VARCHAR(20) NOT NULL CHECK (etat IN ('invite','actif','inactif')),
    identifiant_public  VARCHAR(40) NOT NULL UNIQUE,
    invitation_hash     TEXT UNIQUE,
    invitation_expire_le TIMESTAMPTZ,
    version             INTEGER NOT NULL DEFAULT 1,
    cree_le             TIMESTAMPTZ NOT NULL DEFAULT now(),
    modifie_le          TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_comptes_actif_hash CHECK (etat <> 'actif' OR mot_de_passe_hash IS NOT NULL),
    CONSTRAINT chk_comptes_invitation CHECK (invitation_hash IS NULL OR invitation_expire_le IS NOT NULL)
);
CREATE UNIQUE INDEX uq_comptes_email ON comptes (lower(email));
```

RG01 : email unique insensible à la casse. mot_de_passe_hash NULL pendant l'invitation uniquement ; actif exige un hash ; empreinte d'invitation unique avec expiration obligatoire (CDC §6.6).

3.3. Table PERSONNEL

Profil d'agent (lié 1-1 à un compte). L'état et le rôle restent dans comptes.

1. Représentation MPD (texte brut)

```
+-----------------------------------------+
|                PERSONNEL                |
+-----------------------------------------+
| PK compte_id             : BIGINT       |
|    nom_affiche           : VARCHAR(120) |
|    fonction              : VARCHAR(100) |
|    version               : INTEGER      |
|    cree_le               : TIMESTAMPTZ  |
|    modifie_le            : TIMESTAMPTZ  |
+-----------------------------------------+
```

2. Script SQL

```
CREATE TABLE personnel (
    compte_id             BIGINT PRIMARY KEY REFERENCES comptes(id),
    nom_affiche           VARCHAR(120) NOT NULL CHECK (char_length(btrim(nom_affiche)) > 0),
    fonction              VARCHAR(100),
    version               INTEGER NOT NULL DEFAULT 1,
    cree_le               TIMESTAMPTZ NOT NULL DEFAULT now(),
    modifie_le            TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

fonction : facultative, sans effet sur les droits (CDC §6.6).

3.4. Table VOYAGEURS

Profil public du voyageur (lié 1-1 à un compte), distinct de ses coordonnées privées.

1. Représentation MPD (texte brut)

```
+-----------------------------------------+
|               VOYAGEURS                 |
+-----------------------------------------+
| PK compte_id             : BIGINT       |
|    pseudo                : VARCHAR(60)  |
|    avatar_ref            : VARCHAR(120) |
|    zone_libre            : VARCHAR(120) |
|    version               : INTEGER      |
|    cree_le               : TIMESTAMPTZ  |
|    modifie_le            : TIMESTAMPTZ  |
+-----------------------------------------+
```

2. Script SQL

```
CREATE TABLE voyageurs (
    compte_id             BIGINT PRIMARY KEY REFERENCES comptes(id),
    pseudo                VARCHAR(60) NOT NULL CHECK (char_length(btrim(pseudo)) > 0),
    avatar_ref            VARCHAR(120),
    zone_libre            VARCHAR(120),
    version               INTEGER NOT NULL DEFAULT 1,
    cree_le               TIMESTAMPTZ NOT NULL DEFAULT now(),
    modifie_le            TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX uq_voyageurs_pseudo ON voyageurs (lower(pseudo));
```

RG01 : pseudo unique sur lower(pseudo). zone_libre est facultatif et jamais utilisé comme géométrie (CDC §6.7).

3.5. Table SESSIONS

Sessions serveur du personnel et des voyageurs. Le jeton brut n'est jamais stocké.

1. Représentation MPD (texte brut)

```
+-----------------------------------------+
|                SESSIONS                 |
+-----------------------------------------+
| PK id                    : UUID         |
| FK compte_id             : BIGINT       |
|    empreinte_jeton       : TEXT         |
|    cree_le               : TIMESTAMPTZ  |
|    expire_le             : TIMESTAMPTZ  |
|    revoque_le            : TIMESTAMPTZ  |
+-----------------------------------------+
```

2. Script SQL

```
CREATE TABLE sessions (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    compte_id           BIGINT NOT NULL REFERENCES comptes(id),
    empreinte_jeton     TEXT NOT NULL UNIQUE,
    cree_le             TIMESTAMPTZ NOT NULL DEFAULT now(),
    expire_le           TIMESTAMPTZ NOT NULL,
    revoque_le          TIMESTAMPTZ,
    CONSTRAINT chk_sessions_expire CHECK (expire_le > cree_le)
);
```

Table ajoutée (absente de la v0.2) : prévue au CDC §6.1 et §6.7. Pas de colonnes communes (version) pour cette table dans le CDC.

3.6. Table LIGNES

Référentiel partagé des lignes de bus.

1. Représentation MPD (texte brut)

```
+---------------------------------------------+
|                    LIGNES                   |
+---------------------------------------------+
| PK id                  : BIGINT (identity)  |
|    code                : VARCHAR(20)        |
|    libelle             : VARCHAR(150)       |
|    actif               : BOOLEAN            |
|    archive_le          : TIMESTAMPTZ        |
|    version             : INTEGER            |
|    cree_le             : TIMESTAMPTZ        |
|    modifie_le          : TIMESTAMPTZ        |
+---------------------------------------------+
```

2. Script SQL

```
CREATE TABLE lignes (
    id                  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    code                VARCHAR(20) NOT NULL UNIQUE,
    libelle             VARCHAR(150) NOT NULL CHECK (char_length(btrim(libelle)) > 0),
    actif               BOOLEAN NOT NULL DEFAULT TRUE,
    archive_le          TIMESTAMPTZ,
    version             INTEGER NOT NULL DEFAULT 1,
    cree_le             TIMESTAMPTZ NOT NULL DEFAULT now(),
    modifie_le          TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

3.7. Table BUS

Véhicules et état d'exploitation. La ligne et la direction sont obtenues depuis affectations_bus.

1. Représentation MPD (texte brut)

```
+---------------------------------------------+
|                     BUS                     |
+---------------------------------------------+
| PK id                  : BIGINT (identity)  |
|    plaque              : VARCHAR(25)        |
|    etat                : VARCHAR(20)        |
|    actif               : BOOLEAN            |
|    archive_le          : TIMESTAMPTZ        |
|    version             : INTEGER            |
|    cree_le             : TIMESTAMPTZ        |
|    modifie_le          : TIMESTAMPTZ        |
+---------------------------------------------+
```

2. Script SQL

```
CREATE TABLE bus (
    id                  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    plaque              VARCHAR(25) NOT NULL,
    etat                VARCHAR(20) NOT NULL CHECK (etat IN ('actif','inactif','en_panne')),
    actif               BOOLEAN NOT NULL DEFAULT TRUE,
    archive_le          TIMESTAMPTZ,
    version             INTEGER NOT NULL DEFAULT 1,
    cree_le             TIMESTAMPTZ NOT NULL DEFAULT now(),
    modifie_le          TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX uq_bus_plaque
    ON bus (upper(regexp_replace(plaque, '\s', '', 'g')));
```

RG01 : plaque unique après normalisation (majuscules, espaces retirés).

3.8. Table ARRETS - PostGIS

Arrêts géoréférencés. Type geometry(Point, 4326).

1. Représentation MPD (texte brut)

```
+-------------------------------------------------------+
|                        ARRETS                         |
+-------------------------------------------------------+
| PK id                    : BIGINT (identity)          |
|    code                  : VARCHAR(30)                |
|    nom                   : VARCHAR(150)               |
|    position              : geometry  -- Point, 4326   |
|    actif                 : BOOLEAN                    |
|    archive_le            : TIMESTAMPTZ                |
|    version               : INTEGER                    |
|    cree_le               : TIMESTAMPTZ                |
|    modifie_le            : TIMESTAMPTZ                |
+-------------------------------------------------------+
```

2. Script SQL

```
CREATE TABLE arrets (
    id                    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    code                  VARCHAR(30) NOT NULL UNIQUE,
    nom                   VARCHAR(150) NOT NULL CHECK (char_length(btrim(nom)) > 0),
    position              geometry(Point, 4326) NOT NULL,
    actif                 BOOLEAN NOT NULL DEFAULT TRUE,
    archive_le            TIMESTAMPTZ,
    version               INTEGER NOT NULL DEFAULT 1,
    cree_le               TIMESTAMPTZ NOT NULL DEFAULT now(),
    modifie_le            TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_arrets_bornes CHECK (ST_X(position) BETWEEN -180 AND 180
                                    AND ST_Y(position) BETWEEN -90 AND 90)
);
CREATE INDEX idx_arrets_position ON arrets USING GIST (position);
```

RG06 : SRID 4326, X = longitude, Y = latitude, bornes vérifiées. Index GIST obligatoire.

3.9. Table TRAJETS

Itinéraire ordonné d'une ligne. Chaque sens ou variante est un trajet distinct.

1. Représentation MPD (texte brut)

```
+-----------------------------------------------+
|                   TRAJETS                     |
+-----------------------------------------------+
| PK id                    : BIGINT (identity)  |
|    code                  : VARCHAR(30)        |
| FK ligne_id              : BIGINT             |
|    nom                   : VARCHAR(180)       |
|    direction             : VARCHAR(180)       |
|    actif                 : BOOLEAN            |
|    archive_le            : TIMESTAMPTZ        |
|    version               : INTEGER            |
|    cree_le               : TIMESTAMPTZ        |
|    modifie_le            : TIMESTAMPTZ        |
+-----------------------------------------------+
```

2. Script SQL

```
CREATE TABLE trajets (
    id                    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    code                  VARCHAR(30) NOT NULL UNIQUE,
    ligne_id              BIGINT NOT NULL REFERENCES lignes(id),
    nom                   VARCHAR(180) NOT NULL,
    direction             VARCHAR(180) NOT NULL,
    actif                 BOOLEAN NOT NULL DEFAULT FALSE,
    archive_le            TIMESTAMPTZ,
    version               INTEGER NOT NULL DEFAULT 1,
    cree_le               TIMESTAMPTZ NOT NULL DEFAULT now(),
    modifie_le            TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

actif vaut FALSE par défaut tant que les étapes ne sont pas valides (RG04).

3.10. Table TRAJET_ARRETS

Étapes ordonnées d'un trajet. PK composite (trajet_id, ordre). Un arrêt peut être répété (boucle).

1. Représentation MPD (texte brut)

```
+-------------------------------------+
|            TRAJET_ARRETS            |
+-------------------------------------+
| PK trajet_id             : BIGINT   |
| PK ordre                 : INTEGER  |
| FK arret_id              : BIGINT   |
|    minutes_depuis_depart : INTEGER  |
+-------------------------------------+
```

2. Script SQL

```
CREATE TABLE trajet_arrets (
    trajet_id             BIGINT NOT NULL REFERENCES trajets(id),
    ordre                 INTEGER NOT NULL,
    arret_id              BIGINT NOT NULL REFERENCES arrets(id),
    minutes_depuis_depart INTEGER,
    PRIMARY KEY (trajet_id, ordre),
    CONSTRAINT chk_ordre_positif CHECK (ordre > 0),
    CONSTRAINT chk_minutes_positives CHECK (minutes_depuis_depart IS NULL
                                    OR minutes_depuis_depart >= 0)
);
```

minutes_depuis_depart : valeur indicative >= 0 si disponible.

3.11. Table ZONES - PostGIS

Contours réels de couverture. Type geometry(Polygon, 4326).

1. Représentation MPD (texte brut)

```
+-------------------------------------------------------+
|                        ZONES                          |
+-------------------------------------------------------+
| PK id                    : BIGINT (identity)          |
|    code                  : VARCHAR(30)                |
|    nom                   : VARCHAR(150)               |
|    contour               : geometry  -- Polygon, 4326 |
|    actif                 : BOOLEAN                    |
|    archive_le            : TIMESTAMPTZ                |
|    version               : INTEGER                    |
|    cree_le               : TIMESTAMPTZ                |
|    modifie_le            : TIMESTAMPTZ                |
+-------------------------------------------------------+
```

2. Script SQL

```
CREATE TABLE zones (
    id                    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    code                  VARCHAR(30) NOT NULL UNIQUE,
    nom                   VARCHAR(150) NOT NULL CHECK (char_length(btrim(nom)) > 0),
    contour               geometry(Polygon, 4326) NOT NULL,
    actif                 BOOLEAN NOT NULL DEFAULT FALSE,
    archive_le            TIMESTAMPTZ,
    version               INTEGER NOT NULL DEFAULT 1,
    cree_le               TIMESTAMPTZ NOT NULL DEFAULT now(),
    modifie_le            TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_zones_contour CHECK (ST_IsValid(contour)
                                    AND NOT ST_IsEmpty(contour) AND ST_Area(contour) > 0)
);
CREATE INDEX idx_zones_contour ON zones USING GIST (contour);
```

RG06 : contour valide, non vide, surface positive (au moins 3 sommets distincts). actif vaut FALSE par défaut jusqu'à validation géographique.

3.12. Table AFFECTATIONS_BUS

Historique de la relation bus - trajet. Intervalles [début, fin[.

1. Représentation MPD (texte brut)

```
+-----------------------------------------------+
|                AFFECTATIONS_BUS               |
+-----------------------------------------------+
| PK id                    : BIGINT (identity)  |
| FK bus_id                : BIGINT             |
| FK trajet_id             : BIGINT             |
|    debut_le              : TIMESTAMPTZ        |
|    fin_le                : TIMESTAMPTZ        |
| FK cree_par              : BIGINT             |
|    cree_le               : TIMESTAMPTZ        |
+-----------------------------------------------+
```

2. Script SQL

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
CREATE TABLE affectations_bus (
    id                   BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    bus_id               BIGINT NOT NULL REFERENCES bus(id),
    trajet_id            BIGINT NOT NULL REFERENCES trajets(id),
    debut_le             TIMESTAMPTZ NOT NULL,
    fin_le               TIMESTAMPTZ,
    cree_par             BIGINT NOT NULL REFERENCES personnel(compte_id),
    cree_le              TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_periode CHECK (fin_le IS NULL OR fin_le > debut_le),
    CONSTRAINT excl_affectation_chevauchement
        EXCLUDE USING gist (bus_id WITH =, tstzrange(debut_le, fin_le, '[)') WITH &&)
);
```

RG05 : pas de périodes superposées pour un même bus (contrainte d'exclusion, extension btree_gist à inclure dans les migrations). Fin NULL = borne supérieure ouverte.

3.13. Table SIGNALEMENTS

Contribution d'un voyageur concernant un bus.

1. Représentation MPD (texte brut)

```
+-----------------------------------------------+
|                SIGNALEMENTS                   |
+-----------------------------------------------+
| PK id                    : BIGINT (identity)  |
|    reference             : VARCHAR(30)        |
| FK auteur_id             : BIGINT             |
| FK bus_id                : BIGINT             |
|    type                  : VARCHAR(30)        |
|    commentaire           : VARCHAR(600)       |
|    statut                : VARCHAR(25)        |
|    version               : INTEGER            |
|    cree_le               : TIMESTAMPTZ        |
+-----------------------------------------------+
```

2. Script SQL

```
CREATE TABLE signalements (
    id                   BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    reference            VARCHAR(30) NOT NULL UNIQUE,
    auteur_id            BIGINT NOT NULL REFERENCES voyageurs(compte_id),
    bus_id               BIGINT NOT NULL REFERENCES bus(id),
    type                 VARCHAR(30) NOT NULL,
    commentaire          VARCHAR(600),
    statut               VARCHAR(25) NOT NULL,
    version              INTEGER NOT NULL DEFAULT 1,
    cree_le              TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_signalements_type CHECK (type IN ('en_panne','autre')),
    CONSTRAINT chk_signalements_statut CHECK (statut IN ('a_verifier','confirme','classe_sans_suite'))
);
CREATE INDEX idx_signalements_statut ON signalements (statut, cree_le);
```

RG07 : la décision d'un signalement ne modifie jamais bus.etat. Le type « autre » est extensible par règle validée.

3.14. Table HISTORIQUE_SIGNALEMENTS

Événements de traitement immuables.

1. Représentation MPD (texte brut)

```
+----------------------------------------------+
|            HISTORIQUE_SIGNALEMENTS           |
+----------------------------------------------+
| PK id                    : BIGINT (identity) |
| FK signalement_id        : BIGINT            |
| FK agent_id              : BIGINT            |
|    ancien_statut         : VARCHAR(25)       |
|    nouveau_statut        : VARCHAR(25)       |
|    motif                 : TEXT              |
|    cree_le               : TIMESTAMPTZ       |
+----------------------------------------------+
```

2. Script SQL

```
CREATE TABLE historique_signalements (
    id                    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    signalement_id        BIGINT NOT NULL REFERENCES signalements(id),
    agent_id              BIGINT NOT NULL REFERENCES personnel(compte_id),
    ancien_statut         VARCHAR(25) NOT NULL,
    nouveau_statut        VARCHAR(25) NOT NULL,
    motif                 TEXT NOT NULL CHECK (char_length(btrim(motif)) > 0),
    cree_le               TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_hs_ancien CHECK (ancien_statut IN ('a_verifier','confirme','classe_sans_suite')),
    CONSTRAINT chk_hs_nouveau CHECK (nouveau_statut IN ('a_verifier','confirme','classe_sans_suite'))
);
CREATE INDEX idx_hist_signalements_signalement ON historique_signalements (signalement_id);
```

3.15. Table AVIS

Avis associé à un trajet existant.

1. Représentation MPD (texte brut)

```
+----------------------------------------------+
|                     AVIS                     |
+----------------------------------------------+
| PK id                    : BIGINT (identity) |
|    reference             : VARCHAR(30)       |
| FK auteur_id             : BIGINT            |
| FK trajet_id             : BIGINT            |
|    texte                 : VARCHAR(600)      |
|    statut                : VARCHAR(20)       |
|    cree_le               : TIMESTAMPTZ       |
|    modifie_le            : TIMESTAMPTZ       |
|    version               : INTEGER           |
+----------------------------------------------+
```

2. Script SQL

```
CREATE TABLE avis (
    id                    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    reference             VARCHAR(30) NOT NULL UNIQUE,
    auteur_id             BIGINT NOT NULL REFERENCES voyageurs(compte_id),
    trajet_id             BIGINT NOT NULL REFERENCES trajets(id),
    texte                 VARCHAR(600) NOT NULL,
    statut                VARCHAR(20) NOT NULL,
    cree_le               TIMESTAMPTZ NOT NULL DEFAULT now(),
    modifie_le            TIMESTAMPTZ NOT NULL DEFAULT now(),
    version               INTEGER NOT NULL DEFAULT 1,
    CONSTRAINT chk_avis_texte CHECK (char_length(btrim(texte)) BETWEEN 1 AND 600),
    CONSTRAINT chk_avis_statut CHECK (statut IN ('a_verifier','publie','masque'))
);
CREATE INDEX idx_avis_statut ON avis (statut, cree_le);
```

RG08 : seuls les avis publie sont publics ; la cible (trajet) reste stable lors d'une modération.

3.16. Table HISTORIQUE_AVIS

Décisions de modération immuables. Ne contient pas de nouvelle cible de trajet.

1. Représentation MPD (texte brut)

```
+-----------------------------------------------+
|               HISTORIQUE_AVIS                 |
+-----------------------------------------------+
| PK id                    : BIGINT (identity)  |
| FK avis_id               : BIGINT             |
| FK agent_id              : BIGINT             |
|    ancien_statut         : VARCHAR(20)        |
|    nouveau_statut        : VARCHAR(20)        |
|    motif                 : TEXT               |
|    cree_le               : TIMESTAMPTZ        |
+-----------------------------------------------+
```

2. Script SQL

```
CREATE TABLE historique_avis (
    id                    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    avis_id               BIGINT NOT NULL REFERENCES avis(id),
    agent_id              BIGINT NOT NULL REFERENCES personnel(compte_id),
    ancien_statut         VARCHAR(20) NOT NULL,
    nouveau_statut        VARCHAR(20) NOT NULL,
    motif                 TEXT NOT NULL CHECK (char_length(btrim(motif)) > 0),
    cree_le               TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_ha_ancien CHECK (ancien_statut IN ('a_verifier','publie','masque')),
    CONSTRAINT chk_ha_nouveau CHECK (nouveau_statut IN ('a_verifier','publie','masque'))
);
CREATE INDEX idx_hist_avis_avis ON historique_avis (avis_id);
```

3.17. Table MOUVEMENTS_POINTS

Registre append-only. Solde = SUM(variation). Idempotence via UUID.

1. Représentation MPD (texte brut)

```
+-----------------------------------------------+
|              MOUVEMENTS_POINTS                |
+-----------------------------------------------+
| PK id                    : BIGINT (identity)  |
| FK voyageur_id           : BIGINT             |
| FK agent_id              : BIGINT             |
|    variation             : INTEGER            |
|    motif                 : TEXT               |
|    cle_idempotence       : UUID               |
| FK compense_id           : BIGINT             |
|    cree_le               : TIMESTAMPTZ        |
+-----------------------------------------------+
```

2. Script SQL

```
CREATE TABLE mouvements_points (
    id                    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    voyageur_id           BIGINT NOT NULL REFERENCES voyageurs(compte_id),
    agent_id              BIGINT NOT NULL REFERENCES personnel(compte_id),
    variation             INTEGER NOT NULL,
    motif                 TEXT NOT NULL CHECK (char_length(btrim(motif)) > 0),
    cle_idempotence       UUID NOT NULL UNIQUE,
    compense_id           BIGINT UNIQUE REFERENCES mouvements_points(id),
    cree_le               TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_variation_non_nulle CHECK (variation <> 0)
);
CREATE INDEX idx_mouvements_voyageur ON mouvements_points (voyageur_id, cree_le);
```

RG09 : pas de modification ni suppression d'écriture passée ; correction = mouvement compensateur unique (compense_id UNIQUE), même bénéficiaire, variation opposée.

3.18. Table JOURNAL_AUDIT

Trace technique et administrative des opérations sensibles. Immuable.

1. Représentation MPD (texte brut)

```
+-----------------------------------------------+
|                JOURNAL_AUDIT                  |
+-----------------------------------------------+
| PK id                    : BIGINT (identity)  |
| FK acteur_id             : BIGINT             |
|    action                : VARCHAR(60)        |
|    table_cible           : VARCHAR(60)        |
|    cle_cible             : TEXT               |
|    avant                 : JSONB              |
|    apres                 : JSONB              |
|    requete_id            : UUID               |
|    cree_le               : TIMESTAMPTZ        |
+-----------------------------------------------+
```

2. Script SQL

```
CREATE TABLE journal_audit (
    id                  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    acteur_id           BIGINT NOT NULL REFERENCES comptes(id),
    action              VARCHAR(60) NOT NULL,
    table_cible         VARCHAR(60) NOT NULL,
    cle_cible           TEXT NOT NULL,
    avant               JSONB,
    apres               JSONB,
    requete_id          UUID NOT NULL,
    cree_le             TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

action / table_cible : « liste d'actions contrôlée » au CDC, mais la liste n'est pas fournie ; le CHECK sera ajouté une fois la liste validée. cle_cible n'est pas une FK polymorphe (RG, §6.12).

4. Éléments PostGIS

• Type retenu : **geometry** avec SRID **4326** (WGS 84). Extension : CREATE EXTENSION postgis.

• Convention : X = longitude, Y = latitude (CDC §4.1).

• Index spatiaux : type **GIST** sur arrets.position et zones.contour.

• Validation : bornes des points (arrets) ; contour valide, non vide, surface positive (zones) - RG06.

• Extension btree_gist : requise par la contrainte d'exclusion de affectations_bus (RG05).

5. Index et vues

Index prioritaires (CDC §6.13) : index uniques normalisés (email, pseudo, plaque), GiST sur arrets.position et zones.contour, signalements(statut, cree_le), avis(statut, cree_le), mouvements_points(voyageur_id, cree_le), affectations_bus(bus_id) WHERE fin_le IS NULL. Ils sont dans les scripts de la section 3.

Index sur clés étrangères (« indexer les FK utilisées en jointure », CDC §6.13) :

```
CREATE INDEX idx_comptes_role            ON comptes (role_id);
CREATE INDEX idx_sessions_compte         ON sessions (compte_id);
CREATE INDEX idx_trajets_ligne           ON trajets (ligne_id);
CREATE INDEX idx_trajet_arrets_arret     ON trajet_arrets (arret_id);
CREATE INDEX idx_affectations_bus        ON affectations_bus (bus_id) WHERE fin_le IS NULL;
CREATE INDEX idx_affectations_trajet     ON affectations_bus (trajet_id);
CREATE INDEX idx_signalements_auteur     ON signalements (auteur_id);
CREATE INDEX idx_signalements_bus        ON signalements (bus_id);
CREATE INDEX idx_avis_auteur             ON avis (auteur_id);
CREATE INDEX idx_avis_trajet             ON avis (trajet_id);
CREATE INDEX idx_hist_signalements_agent ON historique_signalements (agent_id);
CREATE INDEX idx_hist_avis_agent         ON historique_avis (agent_id);
CREATE INDEX idx_mouvements_agent        ON mouvements_points (agent_id);
CREATE INDEX idx_journal_acteur          ON journal_audit (acteur_id);
```

Vues attendues (CDC §6.13) : soldes, indicateurs du tableau de bord (définition BF02), coordonnées d'arrêts, affectations courantes.

```
CREATE VIEW v_soldes_voyageurs AS
    SELECT v.compte_id AS voyageur_id,
           COALESCE(SUM(m.variation), 0) AS solde
    FROM voyageurs v
    LEFT JOIN mouvements_points m ON m.voyageur_id = v.compte_id
    GROUP BY v.compte_id;

CREATE VIEW v_coordonnees_arrets AS
    SELECT id, code, nom,
           ST_Y(position) AS latitude,
           ST_X(position) AS longitude
    FROM arrets;

CREATE VIEW v_affectations_courantes AS
    SELECT *
    FROM affectations_bus
    WHERE debut_le <= now() AND (fin_le IS NULL OR fin_le > now());

CREATE VIEW v_indicateurs_tableau_de_bord AS
    SELECT
      (SELECT count(*) FROM bus
        WHERE actif AND etat = 'actif' AND archive_le IS NULL)  AS bus_en_service,
      (SELECT count(*) FROM zones
        WHERE actif AND archive_le IS NULL)                     AS zones_actives,
      (SELECT count(*) FROM comptes c JOIN roles r ON r.id = c.role_id
        WHERE r.code = 'voyageur')                              AS comptes_voyageurs,
      (SELECT count(*) FROM signalements
        WHERE statut = 'a_verifier')                            AS signalements_a_verifier;
```

6. Choix techniques principaux

• **BIGINT GENERATED ALWAYS AS IDENTITY** pour les identifiants techniques ; SMALLINT pour roles.id (CDC §6.2 et §6.6).

• **Archive** via archive_le (timestamptz NULL) pour lignes, bus, arrets, trajets, zones - RG10/RG11.

• **version** pour concurrence optimiste sur comptes, personnel, voyageurs, lignes, bus, arrets, trajets, zones, signalements, avis - RG12.

• **mouvements_points** en mode append-only + clé d'idempotence UUID (RG09).

• **Clés étrangères** sans cascade (RESTRICT par défaut, RG10).

• **TIMESTAMPTZ** stocké en UTC.

7. Journal des corrections (v0.2 vers v0.3)

Table	v0.2	v0.3 (CDC)	Réf. CDC
(liste)	parametres (non prévue)	Supprimée	§6.1
(liste)	sessions absente	Table ajoutée	§6.1, §6.7
(ordre)	comptes avant roles	roles en premier (la FK role_id l'exige)	§6.1
(toutes)	BIGSERIAL	BIGINT GENERATED ALWAYS AS IDENTITY	§6.2
roles	id BIGSERIAL ; libelle(100) ; description	id SMALLINT ; libelle(80) ; sans description ; CHECK sur code	§6.6
comptes	mot_de_passe VARCHAR(255) NN ; actif	mot_de_passe_hash TEXT NULL ; etat (invite/actif/inactif) ; actif exige un hash	§6.6
comptes	role_id absent	role_id SMALLINT NN FK roles(id)	§6.6
comptes	-	identifiant_public NN UQ ; invitation_hash UQ ; invitation_expire_le	§6.6
personnel	role_id, prenom, nom, actif	supprimés ; nom_affiche(120) NN non vide ; fonction(100) ; version	§6.6, §6.4
voyageurs	prenom, nom, actif ; pseudo(80) nullable	supprimés ; pseudo(60) NN non vide, UQ lower(pseudo) ; avatar_ref ; zone_libre ; version	§6.7
lignes	nom ; archive BOOLEAN ; sans version	libelle non vide ; archive_le ; version	§6.2, §6.7
bus	code, capacite ; plaque(20) UQ simple ; archive BOOLEAN	supprimés ; plaque(25) UQ normalisé ; archive_le	§6.8, RG01
arrets	actif DEFAULT FALSE ; sans bornes ; archive BOOLEAN	actif DEFAULT TRUE ; CHECK bornes lat/long ; archive_le	§6.8, RG06
trajets / zones	archive BOOLEAN	archive_le timestamptz NULL	§6.2

Table	v0.2	v0.3 (CDC)	Réf. CDC
zones	validation en commentaire	CHECK ST_IsValid, non vide, surface > 0	RG06, CA07
trajet_arrets	sans contrôle des minutes	CHECK minutes_depuis_depart >= 0	§6.9
affectations_bus	anti-chevauchement « à prévoir »	contrainte EXCLUDE (btree_gist) implémentée	RG05, CA06
signalements / avis / historiques	statuts et type libres	CHECK sur statuts, type, longueur du texte d'avis, motif non vide	§4.2, §6.10-6.11
mouvements_points	compense_id sans UNIQUE ; motif non contrôlé	compense_id UNIQUE ; motif non vide	§6.11
(index/vues)	index du §5 absents des scripts ; aucune vue	index et 4 vues dans les scripts	§6.13

8. Points non précisés par le CDC (rien d'inventé)

• **journal_audit.action / table_cible** : le CDC parle d'une « liste d'actions contrôlée » sans la fournir. Aucun CHECK n'est posé tant que la liste n'est pas validée.

• **Noms des vues et des index** : choisis par nécessité technique ; le CDC n'en impose pas.

• **Contrôles à traduire en migrations** (CDC §6.13 : « nécessitent des contrôles supplémentaires ») : RG02 cohérence profil/rôle (contrôle différé), RG03 dernier administrateur actif, RG04 trajet actif (au moins deux étapes, ordre continu), immutabilité des registres (RG09, CA12, CA17), correction compensatrice même bénéficiaire et variation opposée. Le CDC n'en prescrit pas le mécanisme exact ; ils ne sont donc pas codés ici.

• **Règles API** (transitions de statut, RG12 UPDATE conditionné par version) : hors du MPD.